



Write

I Built an AI Data Analyst App from Scratch — Here's How I Taught a Flask App to Think

Updated March 25, 2026 • 13 min read • View as Markdown

DataLens



```
def main():  
    # Initialize the Flask app  
    app = Flask(__name__)
```

```
    # Load the CSV data  
    df = pd.read_csv('data.csv')
```

```
    # Initialize the AI model  
    model = GroqLLaMA3()
```

```
    # Create the dashboard  
    dashboard = DataDashboard(df, model)
```

S Shlok Bam

A full walkthrough of building DataLens — CSV uploads, Groq/Llama AI insights, auto-generated charts, user auth, persistent chat history, and PDF export. Including every bug, every "why is this broken" moment, and every concept explained properly.

Before We Start — Why I Built This

I've been getting into AI APIs lately. And like most people who just discovered that you can call a language model from Python in three lines of code, I immediately wanted to do something actually useful with it.

The idea came from a real frustration. I had a sales CSV with 2,800 rows. I wanted to know which region was performing best, what the trend looked like over time, and whether there was a correlation between deal size and product line. I opened Excel, filtered, aggregated, made a pivot table, screamed internally, and gave up.

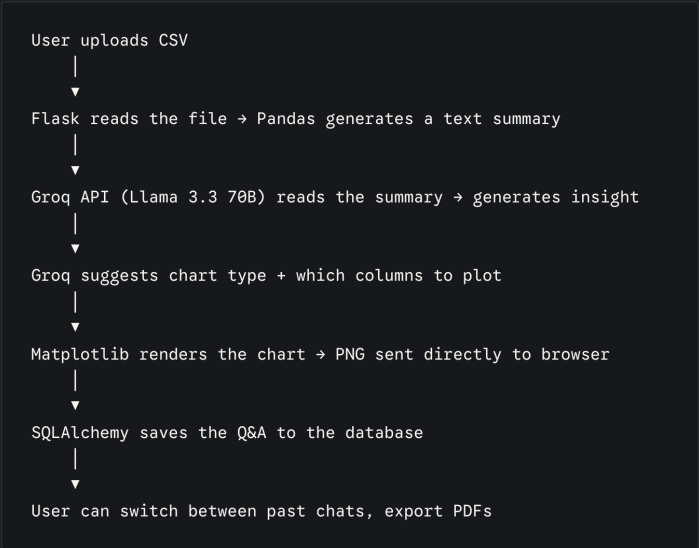
What if I could just *ask* those questions in plain English and get an actual answer?

So I built DataLens — an app where you upload any CSV, ask questions in natural language, get AI-powered insights, and get automatically generated charts. Then I kept going. Added user accounts. Saved conversation history. Added PDF export.

This post covers the full build — every phase, every concept, every error that made me question my choices. If you're learning Flask, SQLAlchemy, or working with AI APIs, there's something here for you.

What I Built

Here's what DataLens does:



Every question you ask is saved. Every analysis session is stored. You can close the tab, come back tomorrow, and pick up exactly where you left off. And when you're done, you can export the whole conversation — questions, AI answers, and charts — as a PDF.

Tech stack:

What	Tool
Web Framework	Flask
Database	SQLAlchemy + SQLite
Auth	Flask-Login

I built this in 4 phases. Let me walk you through each one.



Phase A — SQLAlchemy + Database Design

The first decision was the data model. Three tables:

- **User** — email and hashed password
- **Chat** — each CSV upload creates a Chat (stores the filename and path)
- **Message** — each Q&A exchange is a Message inside a Chat

This is a classic one-to-many relationship:

- One User → many Chats
- One Chat → many Messages

Here's how that looks in SQLAlchemy:

```
class User(db.Model, UserMixin):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(256), nullable=False)
    chats = db.relationship('Chat', backref='user', lazy=True,
                           cascade='all, delete-orphan')

class Chat(db.Model):
    __tablename__ = 'chats'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(200), nullable=False)
    csv_path = db.Column(db.String(500))
    csv_filename = db.Column(db.String(200))
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'))
    messages = db.relationship('Message', backref='chat', lazy=True,
                              cascade='all, delete-orphan')

class Message(db.Model):
    __tablename__ = 'messages'
    id = db.Column(db.Integer, primary_key=True)
    chat_id = db.Column(db.Integer, db.ForeignKey('chats.id'),
                        nullable=False)
    question = db.Column(db.Text, nullable=False)
    answer = db.Column(db.Text, nullable=False)
    chart_type = db.Column(db.String(50))
    chart_x_col = db.Column(db.String(200))
    chart_y_col = db.Column(db.String(200))
```

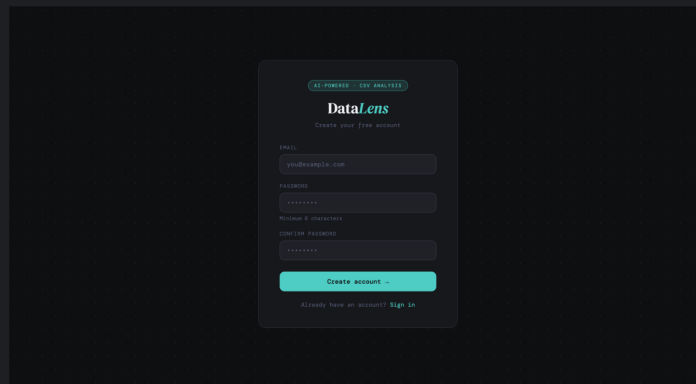
A few things here that are worth understanding:

`cascade='all, delete-orphan'` — when you delete a User, all their Chats get deleted automatically. When you delete a Chat, all its Messages go too. Without this, you'd have orphaned rows sitting in the database forever.

`backref='user'` — this creates a reverse relationship. Once this is set, you can do `chat.user` to get the User who owns that chat, without writing any extra query. SQLAlchemy handles it.

`UserMixin` — Flask-Login needs certain methods on your User model (`is_authenticated`, `get_id()`, etc.). `UserMixin` provides all of these for free. You just inherit from it.

No separate migration tool needed for this project. Just `db.create_all()` inside the app context on startup, and all three tables get created automatically.



Phase B — Flask Blueprints + Auth

This is where I learned what Blueprints actually are, not just theoretically.

A Blueprint is Flask's way of splitting a large app into smaller, reusable pieces. Instead of dumping everything in `app.py`, you put auth-related routes in `auth.py` as a Blueprint and register it in `app.py`. The routes behave identically — they're just organized.

```
# auth.py
from flask import Blueprint, render_template, request, redirect, u
from flask_login import login_user, logout_user, login_required, c

auth_bp = Blueprint('auth', __name__)

@auth_bp.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('index'))

    if request.method == 'POST':
        email = request.form.get('email', '').strip().lower()
        password = request.form.get('password', '')

        user = User.query.filter_by(email=email).first()

        if not user or not user.check_password(password):
            flash('Invalid email or password.', 'error')
            return render_template('login.html')

        login_user(user, remember=True)
        return redirect(url_for('index'))

    return render_template('login.html')
```

```
# app.py
from auth import auth_bp
app.register_blueprint(auth_bp)
```

That's it. The route lives at `/login` and you reference it anywhere as `url_for('auth.login')`. The `'auth.'` prefix is the Blueprint name. One of those things where once you see it, it clicks immediately.

Password hashing — I ran into a compatibility issue here. Werkzeug 2.x defaults to `script` for hashing. But `script` requires OpenSSL compiled with `script` support, and my Python 3.9 environment didn't have it:

```
AttributeError: module 'hashlib' has no attribute 'script'
```

Fix was simple — explicitly specify `pbkdf2:sha256`:

```
def set_password(self, password):
    self.password_hash = generate_password_hash(password, method='pbkdf2:sha256')
```

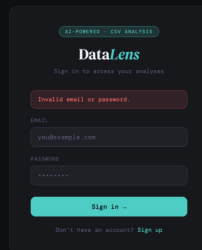
`pbkdf2:sha256` is NIST-approved, used by production apps everywhere, and works on all Python versions. Perfectly fine security-wise.

Protecting routes — one decorator and a route is fully protected:

```
@app.route('/upload', methods=['POST'])
@login_required
def upload_file():
    ...
```

Unauthenticated requests get redirected to the login page automatically. Just make sure you tell Flask-Login where your login page is:

```
login_manager.login_view = 'auth.login'
```



Phase C — Multi-Chat Routing

Here's where it got interesting.

The original version of the app was stateless — you uploaded a file, asked questions, everything lived in the Flask session (basically a browser cookie). Close the tab and it was gone. Not great.

Phase C converts it to full persistence. Every upload creates a Chat row. Every question creates a Message row. The user's sidebar shows all their past analyses.

```
@app.route('/upload', methods=['POST'])
@login_required
def upload_file():
    file = request.files['file']
    filename = secure_filename(file.filename)
    filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
    file.save(filepath)

    # Phase C: create a Chat record in the database
    chat = Chat(
        name=filename.replace('.csv', '').replace('_', ' ').title(
            csv_path=filepath,
            csv_filename=filename,
```

```
msg = Message(
    chat_id      = session.get('chat_id'),
    question     = user_question,
    answer       = insight,
    chart_type   = chart_type,
    chart_x_col  = chart_column_suggestion.get('x'),
    chart_y_col  = chart_column_suggestion.get('y'),
)
db.session.add(msg)
db.session.commit()
```

The chat-switching API has three routes:

```
GET    /chats          # list all chats for current user
GET    /chats/<id>      # get all messages for one chat
POST   /chats/<id>/activate # restore a chat into the session
DELETE /chats/<id>      # delete chat + cascade messages
```

The frontend sidebar calls `/chats` on page load, renders a list, and when you click a past chat it calls `/chats/<id>/activate` (to restore the session filepath) then `/chats/<id>` (to load the messages). Old Q&A cards get re-rendered and charts reload from the CSV.

One thing I discovered: `db.session.get(User, user_id)` is the correct way to look up by primary key in SQLAlchemy 2.x. The old `User.query.get(id)` syntax still works but fires a deprecation warning on every request:

```
LegacyAPIWarning: The Query.get() method is considered legacy
```

Changed it in the Flask-Login user loader and the warnings went away.



DataLens

+ New analysis

RECENT ANALYSES

Sales Data Sample

0 Q&A · Mar 25, 2026

Sales Data Sample

2 Q&A · Mar 25, 2026

Sales Data

1 Q&A · Mar 25, 2026

Phase D — PDF Export with ReportLab

This was the most satisfying phase to build.

ReportLab is a Python library that gives you full programmatic control over PDF layout. No templates, no HTML-to-PDF conversion — you build every element from scratch in Python code.

The mental model is simple: ReportLab has a `story` — a list of `Flowable` objects that get laid out onto pages in order. You build the list, call `doc.build(story)`, and the library handles page breaks, margins, and layout.

```
from reportlab.platypus import (
    SimpleDocTemplate, Paragraph, Spacer,
    Image, HRFlowable, PageBreak
)

buf = io.BytesIO()

doc = SimpleDocTemplate(buf, pagesize=A4,
    leftMargin=25*mm, rightMargin=25*mm,
    topMargin=20*mm, bottomMargin=20*mm)
```

```
story = []

# Title page
story.append(Spacer(1, 30*mm))
story.append(Paragraph('DataLens', title_style))
story.append(Paragraph(chat.name, subtitle_style))
story.append(HRFlowable(width='100%', thickness=1, color=accent_color))
story.append(PageBreak())

# Q&A sections
for msg in messages:
    story.append(Paragraph(msg.question, question_style))
    story.append(Paragraph(msg.answer, answer_style))

    if msg.chart_type != 'none':
        chart_buf = regenerate_chart(msg)
        story.append(Image(chart_buf, ...))

doc.build(story)
buf.seek(0)
return buf
```

The charts are re-generated on-the-fly — I pass a `chart_generator` closure into `build_pdf()` that reads the original CSV and rerenders the chart as a PNG. This is clean because no image bytes are stored in the database.

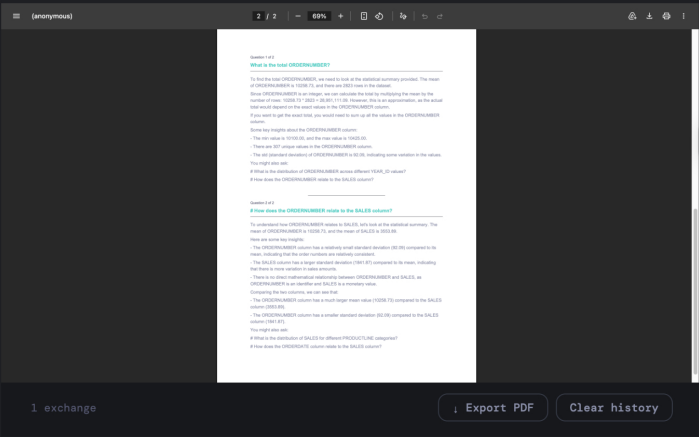
The export route itself is simple:

```
@app.route('/export/<int:chat_id>')
@login_required
def export_pdf(chat_id):
    chat = Chat.query.filter_by(id=chat_id, user_id=current_user.id)

    pdf_buf = build_pdf(chat, list(chat.messages), chart_generator)

    return send_file(pdf_buf,
        mimetype='application/pdf',
        as_attachment=True,
        download_name=f'datalens_{chat.name.lower().replace(" ", " ")}
```

`as_attachment=True` adds `Content-Disposition: attachment` to the response — that's the HTTP header that tells the browser to download the file instead of trying to display it inline.



The AI Part — How It Actually Works

Most of the "magic" is in `gemini_helper.py` (badly named — it actually uses the Groq API, not Google Gemini, but I kept the filename to avoid confusion).

breaking imports).

The key insight: I never send the full CSV to the AI. Sending 2,800 rows to a language model would blow past the context limit, cost tokens, and be slow. Instead, I pre-process the CSV into a compact text summary:

```
Shape: 2823 rows x 25 columns

Column Types:
  Numeric: QUANTITYORDERED, PRICEEACH, SALES, MSRP
  Categorical: STATUS, PRODUCTLINE, COUNTRY, TERRITORY

Statistics (numeric columns):
  SALES: mean=3553.89, std=1841.87, min=482.13, max=14082.80

Top Values:
  PRODUCTLINE: Classic Cars (967), Vintage Cars (607), Motorcycles
  COUNTRY: USA (1004), Spain (342), France (314)

Missing Values: None

Sample Rows:
  ORDERNUMBER  SALES  PRODUCTLINE  COUNTRY
10107          2871   Motorcycles   USA
...
```

This summary — not the raw CSV — gets sent to the AI. It's maybe 800 tokens vs. tens of thousands. The model can answer most analytical questions accurately from this structured summary.

Three separate AI calls happen for each question:

1. `get_ai_insight()` — the main call. Gets the text answer. Includes the last 5 exchanges as context so follow-up questions work properly.
2. `suggest_chart_type()` — a separate call with `temperature=0` (deterministic). Returns exactly one word: `bar`, `line`, `scatter`, `histogram`, `pie`, or `none`. Low temperature because I need a parseable response, not creativity.
3. `suggest_chart_columns()` — another separate call. Returns JSON with `x` and `y` column names. I parse this, validate against the actual column list, and fall back to sensible defaults if the AI hallucinates a column name that doesn't exist.

Why three calls instead of one? When I tried to get everything in one call, the AI would sometimes get distracted and return malformed JSON, or mix the chart suggestion into the text answer. Separating concerns made each call simpler and more reliable.

YOU # How does the ORDERNUMBER relate to the SALES column?

To understand how `ORDERNUMBER` relates to `SALES`, let's look at the statistical summary. The mean of `ORDERNUMBER` is 10258.73, and the mean of `SALES` is 3553.89.

Here are some key insights:

- » The `ORDERNUMBER` column has a relatively small standard deviation (92.09) compared to its mean, indicating that the order numbers are relatively consistent.
- » The `SALES` column has a larger standard deviation (1841.87) compared to its mean, indicating that there is more variation in sales amounts.
- » There is no direct mathematical relationship between `ORDERNUMBER` and `SALES`, as `ORDERNUMBER` is an identifier and `SALES` is a monetary value.

Comparing the two columns, we can see that:

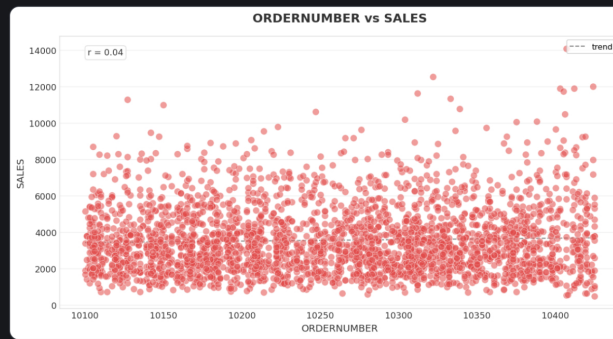
- » The `ORDERNUMBER` column has a much larger mean value (10258.73) compared to the `SALES` column (3553.89).
- » The `ORDERNUMBER` column has a smaller standard deviation (92.09) compared to the `SALES` column (1841.87).

YOU MIGHT ALSO ASK

What is the distribution of SALES for different PRODUCTLINE categories?

How does the ORDERDATE column relate to the SALES column?

AUTO-GENERATED CHART - SCATTER



Everything That Went Wrong — Summary

Problem	Cause	Fix
<code>hashlib</code> has no attribute <code>'script'</code>	Python 3.9 missing <code>script</code> support	Explicitly use <code>method='pbkdf2:sha256'</code> in <code>generate_password_hash</code>
Upload returning 500	CSV with non-UTF-8 characters	<code>try: pd.read_csv(f) except UnicodeDecodeError: pd.read_csv(f, encoding='latin1')</code>
Data preview table blank	Pandas NaN serializes as bare <code>NaN</code> — invalid JSON	<code>df.where(pd.notnull(df), None)</code> before <code>to_dict()</code>
<code>LegacyAPIWarning</code> on every request	<code>User.query.get()</code> deprecated in SQLAlchemy 2.x	Replace with <code>db.session.get(User, user_id)</code>
Auth routes returning 404	Thought Blueprint was at <code>/auth/login</code>	Routes are at <code>/login</code> — no prefix. <code>url_for('auth.login')</code> still works
Chart generator silent failure in PDF	CSV no longer on disk when exporting old chat	Added early check <code>if not os.path.exists(chat.csv_path)</code> before rendering

The NaN one cost me the most time. The symptom was completely confusing — server returned 200, JavaScript got a response, but the table was blank. Silent failure. Turned out `response.json()` was throwing a parse error because `NaN` is not valid JSON (it's `null` in JSON), and the whole preview section was quietly dying in a catch block. Classic.

What I'd Do Differently

1. Proper file storage Right now CSVs are saved to a local `uploads/` folder. If the server restarts, old chat sessions can't reload their charts because the files are gone. In production I'd use S3 — store the CSV path as an S3 key, not a local filesystem path.

2. Background jobs for AI calls Right now the `/ask` endpoint blocks until the AI responds — usually 3-8 seconds. A better pattern is to return a job ID immediately, process the AI call in a background worker (Celery, or even a simple thread), and have the frontend poll or use WebSockets for the result. Feels much faster.

3. Streaming AI responses The Groq API supports streaming responses — you can start sending tokens to the frontend as they arrive, exactly like ChatGPT does. The current setup waits for the full response before returning. Streaming would feel dramatically faster even if total time is the same.

4. PDF charts as stored images Right now the PDF export re-generates charts from the original CSV. If the CSV is gone, charts are skipped silently. Better to store the chart image in S3 alongside the CSV, and reference it directly in the PDF.

Key Takeaways

Send summaries to AI, not raw data. Structured text summaries are more token-efficient, equally informative for analysis, and let you control exactly what context the model has. This is the pattern most production data AI tools use.

Separate your AI calls. One call for the text answer, a separate call for chart type, another for column selection. Each prompt is simpler, outputs are more parseable, and failures are isolated.

SQLAlchemy's cascades are powerful. `cascade='all, delete-orphan'` One time and your entire data hierarchy cleans up automatically. No manual delete queries across tables.

Flask Blueprints are just an organisation. They're not especially complex — they're a way to split a growing `app.py` list into logical groups. Start using them before your app file gets too big, not after.

`NaN` is not `null`. In JSON, missing values are `null`. Python's `float('nan')` serializes to bare `NaN` which browsers can't parse. Always sanitize DataFrames before JSONifying them.

Resources

- GitHub repo: [shlokbam/ai-data-analyst](#)
- [Groq API docs](#)
- [Flask-Login documentation](#)
- [SQLAlchemy ORM tutorial](#)
- [ReportLab user guide](#)

If you're building something similar or hit different errors, drop them in the comments. Always curious what breaks for other people.

👁 10 views

Comments

🗨 Join the discussion

No comments yet. [Be the first to comment.](#)

More from this blog

I Built an AI-Powered Mock Interview Platform from Scratch — Here's Everything That Went Wrong

A full walkthrough of building MockVue — React + FastAPI + TiDB Cloud + Groq AI + face-api.js — including every bug, every architectural decision, and every "why is this not working" moment...

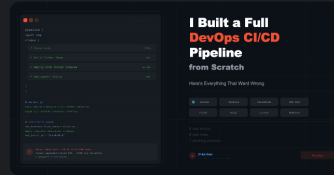
Apr 6, 2026 · 28 min read · 32



I Built a Full DevOps CI/CD Pipeline from Scratch — Here's Everything That Went Wrong

A honest, detailed walkthrough of building a Flask + Docker + Jenkins + Terraform + AWS project — including every error, every fix, and every "why is this not working" moment. Before We Start ...

Mar 14, 2026 · 20 min read · 31



Build. Break. Ship.

3 posts



© 2026 Build. Break. Ship.

[Archive](#) [Privacy](#) [Terms](#)

[Sitemap](#) [RSS](#)

hashnode